

SYSTEM ANALYST

presented by Ayub

ООП: основы для аналитика

Важный контекст: это НЕ урок программирования

ООП — это **способ думать о системе**.

Ты не будешь писать код. Но ты будешь:

- Строить **Class Diagrams** — стандартный артефакт аналитика
- Проектировать **базы данных** (ERD) — классы → таблицы
- Говорить с разработчиками **на одном языке**

Когда разработчик говорит "класс", "объект", "наследование" — ты должен понимать, о чём речь.

Этап 1: Class и Object

Представь: архитектор нарисовал **чертёж квартиры**.

По этому чертежу построили **50 квартир**.

Чертёж — это одно. Квартиры — другое.

В чём разница?

Class (Класс) = чертёж

Class — это шаблон, описание, схема.

Класс определяет:

- Какие **свойства** будут у объекта (имя, email, дата)
- Что объект **умеет делать** (войти, выйти, изменить профиль)

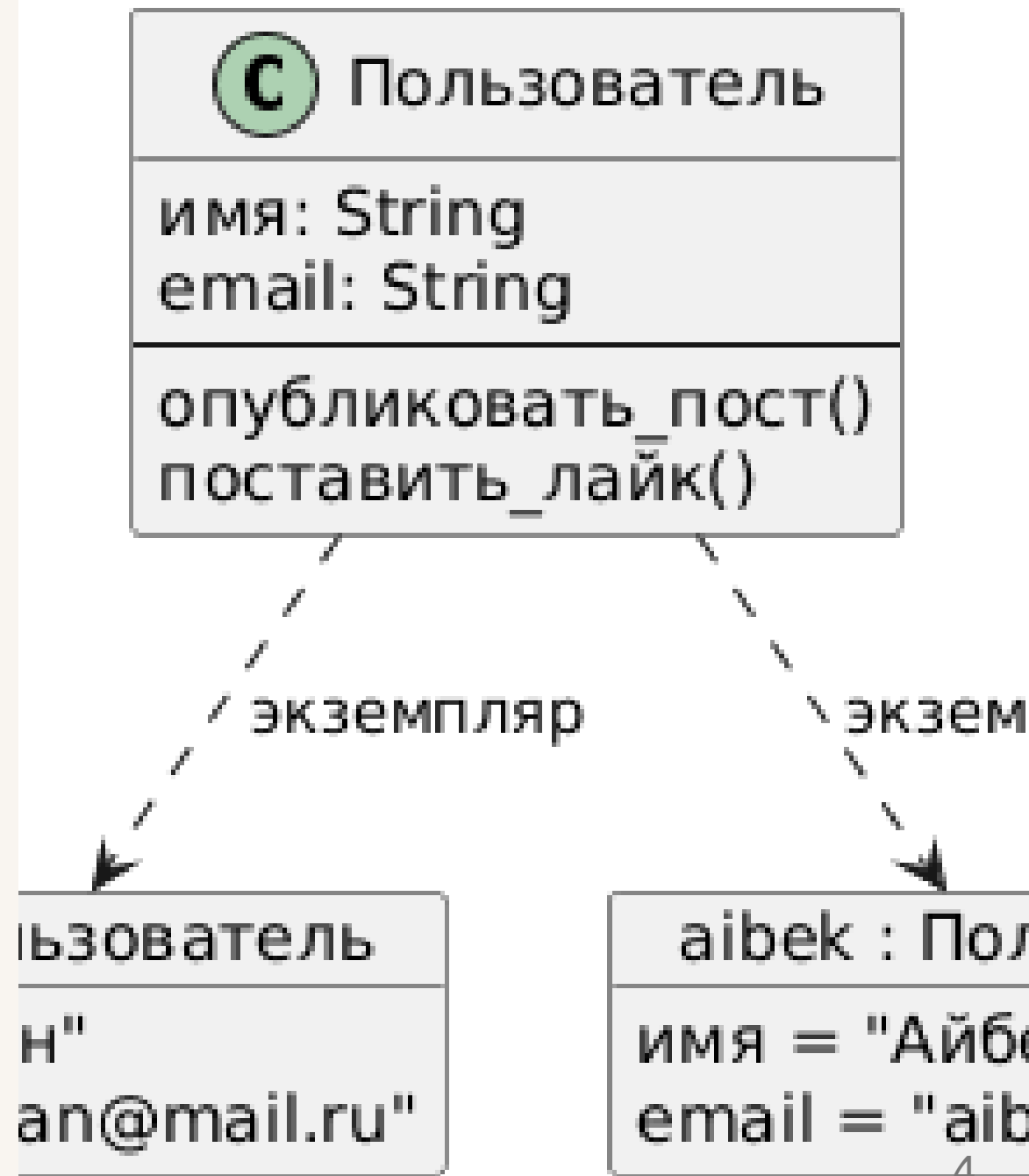
Сам по себе класс **не занимает места** в системе — это просто описание.

Пока нет ни одного пользователя — класс `Пользователь` уже есть.

Object (Объект) = конкретный экземпляр

Object — это реальная "квартира" по чертежу.

- В системе может быть **миллион объектов** одного класса
- Каждый объект — **своя копия** данных
- Все объекты сделаны **по одному шаблону** (классу)



Примеры: Class и Objects в реальных системах

Класс	Объекты (экземпляры)
Пользователь	Айжан, Айбек, Марьям
Товар	iPhone 15, Наушники Sony, Кроссовки Nike
Заказ	Заказ #1001, Заказ #1002, Заказ #1003
Сообщение	"Привет!", "Ты дома?", "Окей"

Один класс → неограниченное количество объектов.

Этап 2: Атрибуты и Методы

Возьмём класс `Пользователь` в Instagram.

Что ЕСТЬ у пользователя? (данные)

Что он УМЕЕТ делать? (действия)

Атрибуты (Attributes) = данные объекта

Атрибуты — это свойства, характеристики. Что **ЕСТЬ** у объекта.

Примеры для класса `Пользователь` в Instagram:

- `имя` — "Айжан Сейткали"
- `email` — "aizhan@mail.ru"
- `аватар` — ссылка на фото
- `дата_регистрации` — "2024-01-15"
- `количество_подписчиков` — 1 240

Методы (Methods) = действия объекта

Методы — это операции, поведение. Что объект **УМЕЕТ** делать.

Примеры для класса `Пользователь` в Instagram:

- `опубликовать_пост()` — создать новую публикацию
- `поставить_лайк()` — лайкнуть пост
- `подписаться()` — подписаться на другого пользователя
- `выйти_из_аккаунта()` — завершить сессию

Структура класса: атрибуты + методы

Пользователь

```
имя: String  
email: String  
аватар: URL  
количество_подписчиков: int
```

```
опубликовать_пост()  
поставить_лайк()  
подписаться()  
выйти_из_аккаунта()
```

Верхняя секция — атрибуты (данные).

Нижняя секция — методы (действия).

Этап 3: Инкапсуляция

Ты пользуешься приложением доставки еды.

Нажимаешь **«Заказать»**.

За кулисами: маршрут считается, курьер назначается, оплата проходит.

Ты видишь, как это работает?

Инкапсуляция (Encapsulation) = скрывание деталей

Инкапсуляция — это принцип "снаружи видно только то, что нужно".

- **+ public** — видно снаружи: кнопки, действия, API
- **- private** — скрыто внутри: алгоритмы, внутренняя логика

Аналогия: кофемашина — ты нажимаешь кнопку, не видишь температуру воды и давление пара.

Зачем аналитику знать про Инкапсуляцию

Инкапсуляция помогает аналитику разграничить:

- `+ public` → что описываем в **API** (что видят другие системы)
- `+ public` → что показываем **пользователю** в интерфейсе
- `- private` → внутренняя логика (SA обычно **не описывает**)



я: String

+ public
(API)

- private
(внут)

Module Check 1

- В чём разница **Class** и **Object**? Объясни своими словами.
- Приведи пример: класс и 3 объекта из Instagram.
- Что такое **атрибуты**? Что такое **методы**?
- Что прячет **Инкапсуляция** — и зачем?

Этап 4: Наследование

Возьмём: **Транспорт, Машина, Электромобиль, Велосипед.**

Как они связаны?

Что у них **общего**? Что **уникальное**?

Наследование (Inheritance) = "является разновидностью"

Наследование — дочерний класс получает всё от родительского + добавляет своё.

"Электромобиль — это **Машина**, но с батареей"

Не нужно повторять: `скорость`, `цвет`, `двигаться()` — всё это уже есть в родителе `Транспорт`.

Стрелка с треугольником в диаграмме = "**является разновидностью**".

Дерево наследования

- Транспорт → Машина →
Электромобиль
- Транспорт → Велосипед

Ребёнок **наследует ВСЁ** от родителя.

Добавляет только то, что уникально для него.

Чем выше по дереву — тем более общая концепция.



Реальный пример наследования для SA

Родитель	Наследник	Что добавляет
Пользователь	Администратор	управлять_пользователями()
Пользователь	Продавец	добавить_товар() , управлять_магазином()
Уведомление	Email	адрес_получателя
Уведомление	Push	токен_устройства

Этап 5: Связи между классами

Студент **записан на** курс.

Машина **СОСТОИТ ИЗ** колёс.

Комната — **часть** здания.

Это разные виды связей. **Чем они отличаются?**

Association (Ассоциация) — слабая связь

Объекты **знают** друг о друге, но **независимы**.

Пример: Студент записан на Курс.

- Студент отчислен → Курс **продолжается**
- Курс закрыт → Студент **учится** на других курсах

Связь существует только пока они взаимодействуют.

Aggregation (Агрегация) — средняя связь

Один объект **является частью** другого, но может **существовать отдельно**.

Пример: Колесо является частью Машины.

- Колесо сняли с машины → оно **не исчезло**
- Его можно поставить на другую машину

Часть **переживает** целое.

Composition (Композиция) — сильная связь

Один объект **является частью** другого и **без него не существует**.

Пример: Комната — часть Здания.

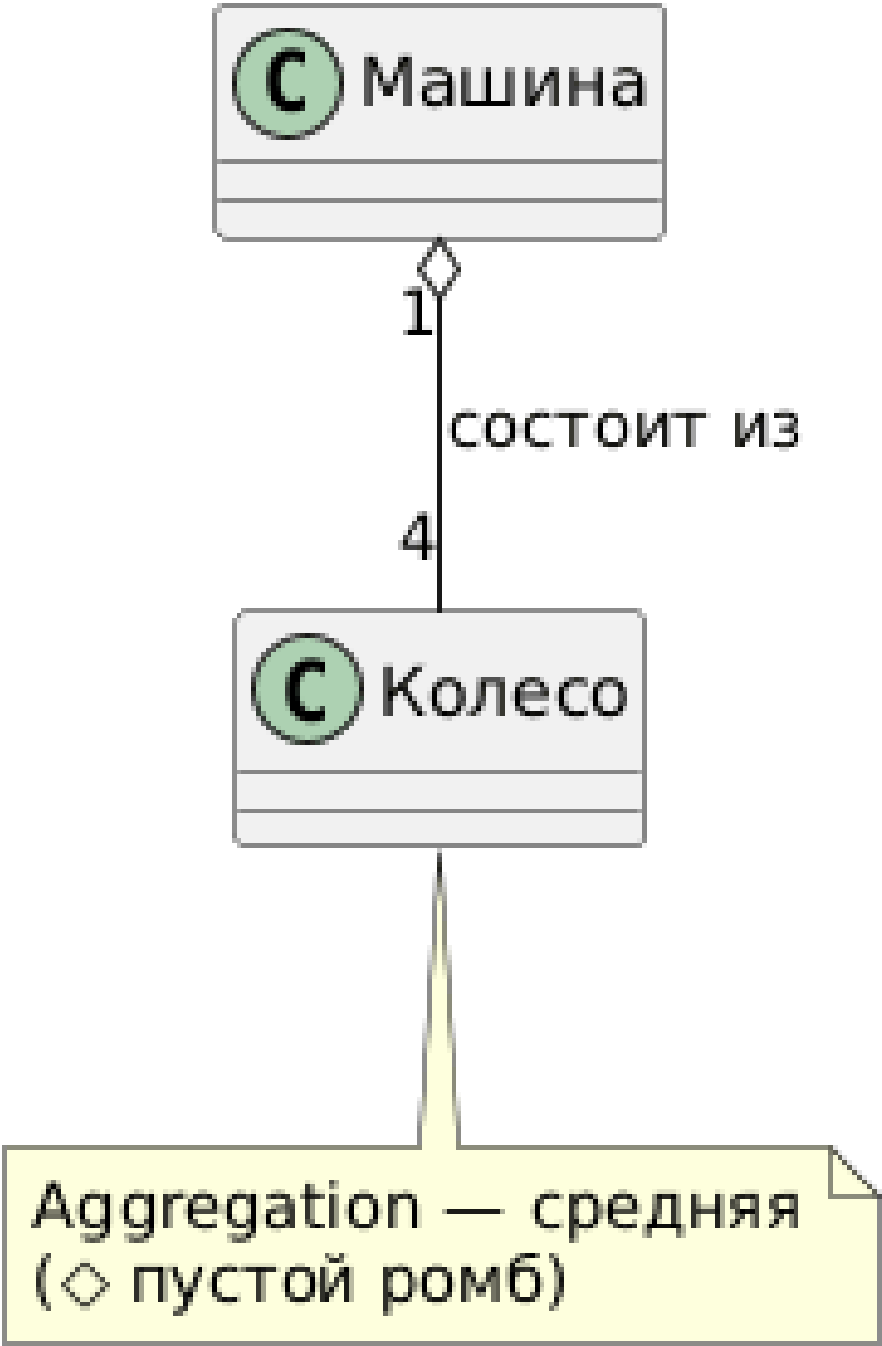
- Здание снесли → комнаты **нет**
- Комната без здания — не существует

Часть **не переживает** целое.

Сравнение трёх типов связей

Тип	Зависимость	Пример
Association	Слабая	Студент ↔ Курс
Aggregation	Средняя	Машина ◇— Колесо
Composition	Сильная	Здание ◆— Комната

Эти же три типа — в **Class Diagram** и **ERD**.



Этап 6: Абстракция

Ты проектируешь систему для **интернет-магазина**.

Класс `Пользователь` .

Нужно ли хранить: `имя` ✓, `email` ✓, **рост пользователя...? Любимый цвет...?**

Почему?

Абстракция (Abstraction) = только нужное

Абстракция — это осознанный выбор: что моделируем, что отбрасываем.

Один и тот же "Пользователь" — разный в разных системах:

- **Интернет-магазин:** имя, email, адрес доставки, история заказов
- **Медицинская система:** ФИО, дата рождения, группа крови, диагнозы
- **HR-система:** должность, отдел, зарплата, дата найма

Контекст **определяет** что важно.

Один и тот же "Пользователь"



Д
О
З
Д

Module Check 2

- Объясни: зачем нужно **Наследование**?
- В чём разница **Aggregation** и **Composition**?
- Класс **Заказ** в интернет-магазине: 5 атрибутов и 3 метода.
- Зачем нужна **Абстракция** при проектировании?

Этап 7: Мост ООП → ERD и SQL

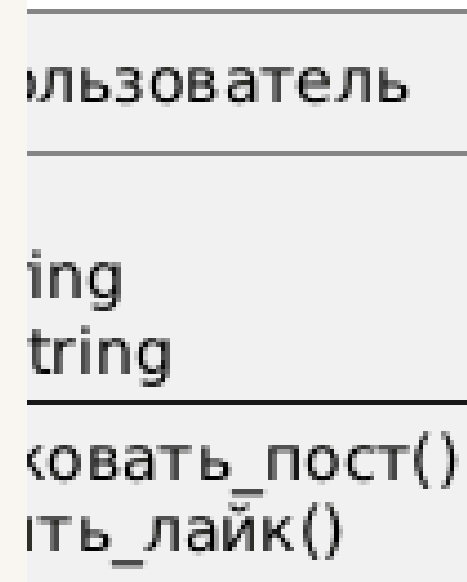
Вот почему аналитику важно понимать ООП — напрямую связано с базами данных:

ООП	База данных
Class	Таблица
Атрибут	Колонка
Object	Строка (запись)
Association	Foreign Key
Inheritance	Отдельная таблица или поле <code>type</code>

Пример: Класс → SQL таблица

- Атрибуты → **колонки** с типами данных
- Методы → в SQL **не хранятся** (логика в коде)
- Связи между классами → **foreign keys**

Когда будем изучать ERD — ты уже будешь знать все концепции.



маппинг



Атрибуты → колонки
Методы → не хранятся
Связи → foreign keys

Итог: что мы разобрали

ООП — это способ думать о системе как о наборе объектов с данными и поведением

- [] **Class** = шаблон, **Object** = конкретный экземпляр
- [] **Атрибуты** = данные объекта, **Методы** = действия объекта
- [] **Инкапсуляция** = прячем детали реализации, показываем интерфейс
- [] **Наследование** = "является разновидностью", не дублируем общее
- [] **Association / Aggregation / Composition** = разная сила связи
- [] **Абстракция** = моделируем только то, что нужно для системы

Что дальше

На следующем уроке — **Class Diagram**.

Ты уже знаешь все концепции.

Class Diagram — это просто **стандартная UML нотация** для тех же идей:

- Классы с атрибутами и методами → прямоугольники с секциями
- Наследование → стрелка с треугольником
- Association / Aggregation / Composition → разные виды линий

Всё, что ты узнал сегодня — будет там.